

Εργασία 2

Μελέτη και κατανόηση των ορίων απόδοσης σε RISC-V επεξεργαστές

Η απόδοση ενός προγράμματος σε έναν επεξεργαστή δεν καθορίζεται μόνο από το πλήθος των εντολών που εκτελούνται, αλλά σε μεγάλο βαθμό από τη μικροαρχιτεκτονική του επεξεργαστή και τον τρόπο με τον οποίο αυτή αλληλεπιδρά με τα χαρακτηριστικά του προγράμματος. Σύγχρονοι επεξεργαστές ενσωματώνουν πολύπλοκους μηχανισμούς, όπως διοχέτευση εντολών (pipelining), εκτέλεση εκτός σειράς (out-of-order execution), πρόβλεψη διακλαδώσεων (branch prediction) και ιεραρχίες κρυφών μνημών (cache hierarchies), με στόχο τη βελτίωση της απόδοσης μέσω της εκμετάλλευσης της παραλληλίας σε επίπεδο εντολής (Instruction-Level Parallelism – ILP) και τη μείωση των καθυστερήσεων από την πρόσβαση στη μνήμη. Ωστόσο, η αποτελεσματικότητα αυτών των μηχανισμών εξαρτάται άμεσα από τη φύση του προγράμματος. Ορισμένα προγράμματα περιέχουν πολλές ανεξάρτητες πράξεις και μπορούν να αξιοποιήσουν πλήρως τις δυνατότητες ενός επεξεργαστή εκτός σειράς, ενώ άλλα χαρακτηρίζονται από ισχυρές εξαρτήσεις δεδομένων ή μη προβλέψιμα μοτίβα πρόσβασης στη μνήμη, περιορίζοντας σημαντικά τα περιθώρια βελτίωσης της απόδοσης. Επομένως, η κατανόηση της απόδοσης απαιτεί ταυτόχρονη μελέτη τόσο της μικροαρχιτεκτονικής όσο και του ίδιου του προγράμματος. Σε αυτή την εργασία, θα μελετήσουμε τη συμπεριφορά δύο διαφορετικών κατηγοριών προγραμμάτων σε επεξεργαστές RISC-V, χρησιμοποιώντας τον μικροαρχιτεκτονικό προσομοιωτή gem5.



Σκοπός της εργασίας και η βασική ιδέα

Σκοπός της εργασίας να συγκρίνουμε την εκτέλεση δυο διαφορετικής φύσης προγραμμάτων σε έναν απλό επεξεργαστή με διοχέτευση εντολών σε σειρά (in-order pipeline) και σε έναν πιο σύνθετο και σύγχρονο επεξεργαστή με διοχέτευση εντολών εκτός σειράς (out-of-order pipeline), προκειμένου να κατανοήσουμε υπό ποιες συνθήκες η εκτέλεση προγραμμάτων σε σχεδιάσεις εκτός σειράς οδηγεί σε σημαντική επιτάχυνση. Παράλληλα, θα εξετάσουμε την επίδραση της ιεραρχίας κρυφών μνημών στην απόδοση, μεταβάλλοντας τα μεγέθη των κρυφών μνημών και παρατηρώντας πως διαφορετικά μοτίβα προσπέλασης στη μνήμη επηρεάζουν τον χρόνο εκτέλεσης. Τέλος, θα μελετήσουμε τον ρόλο βασικών μικροαρχιτεκτονικών δομών ενός επεξεργαστή με διοχέτευση εντολών εκτός σειράς, όπως ο reorder buffer (ROB) και οι φυσικοί καταχωρητές, διερευνώντας πως οι περιορισμοί αυτών των πόρων μπορούν να επηρεάσουν την απόδοση ανάλογα με το πρόγραμμα. Μέσα από τη συστηματική ανάλυση των στατιστικών που παρέχει ο gem5, στόχος της εργασίας είναι να αποκτήσετε μια πιο ουσιαστική κατανόηση της σχέσης μεταξύ προγράμματος και μικροαρχιτεκτονικής, καθώς και των βασικών παραγόντων που καθορίζουν την απόδοση ενός σύγχρονου επεξεργαστή.

Η εργασία εστιάζει σε δύο θεμελιώδη προγράμματα:

1. Ένα πρόγραμμα που αξιοποιεί την παραλληλία επιπέδου εντολής (cpu-intensive program)
2. Ένα πρόγραμμα που περιορίζεται από τις εξαρτήσεις και τα μοτίβα προσπέλασης της μνήμης (memory-intensive program)

Περιβάλλον εκτέλεσης και προγράμματα

Η εργασία θα εκτελεστεί σε αρχιτεκτονική RISC-V, χρησιμοποιώντας τον προσομοιωτή gem5 σε λειτουργία syscall emulation (SE mode – χωρίς την προσομοίωση λειτουργικού συστήματος). Για λόγους ομοιομορφίας και ώστε να διασφαλιστεί ότι όλες οι εκτελέσεις πραγματοποιούνται υπό τις ίδιες συνθήκες, σας δίνεται έτοιμο ένα configuration script σε Python (στο eClass με όνομα `run_riscv_se.py`). Το script αυτό χρησιμοποιείται για τον ορισμό της διαμόρφωσης (π.χ., CPU, κρυφές μνήμες, μεγέθη μικροαρχιτεκτονικών δομών, κλπ.) και σας επιτρέπει να τροποποιείτε εύκολα τις παραμέτρους που θα απαιτηθούν στα επόμενα ερωτήματα της εργασίας.

Στο πλαίσιο της εργασίας, θα μελετήσετε και θα εκτελέσετε τα παρακάτω προγράμματα, τα οποία διατίθενται ως πηγαίοι κώδικες στο eClass:

1. `compute-intensive.c`

Αυτό το πρόγραμμα εκτελεί πολλές ανεξάρτητες αριθμητικές πράξεις μέσα σε έναν βρόχο. Οι πράξεις αυτές δεν εξαρτώνται σημαντικά η μία από την άλλη, άρα υπάρχει περιθώριο για καλύτερη αξιοποίηση της διοχέτευσης και της εκτέλεσης εκτός σειράς.

2. `memory-intensive.c`

Αυτό το πρόγραμμα χρησιμοποιεί τυχαία διάταξη δεδομένων, με αποτέλεσμα να μειώνεται σημαντικά η τοπικότητα μνήμης. Αυτό το καθιστά κατάλληλο για τη μελέτη της επίδρασης του μεγέθους των κρυφών μνημών στην απόδοση.

Λήψη των gem5/RISC-V toolchain & μεταγλώττιση των προγραμμάτων

Από την προηγούμενη εργασία θεωρούμε ότι έχετε ήδη εγκαταστήσει τον gem5 καθώς και το RISC-V toolchain. Σε περίπτωση που δεν τα έχετε διαθέσιμα, μπορείτε να ανατρέξετε στις διαφάνειες της Ενότητας 2 στο eClass, όπου παρέχονται αναλυτικές οδηγίες λήψης και εγκατάστασης.

Στο πλαίσιο της συγκεκριμένης άσκησης, θα πρέπει να μεταγλωττίσετε τα προγράμματα που σας δίνονται από C σε εκτελέσιμα αρχεία για την αρχιτεκτονική RISC-V, χρησιμοποιώντας το εγκατεστημένο RISC-V toolchain. Η μεταγλώττιση μπορεί να γίνει ως εξής:

```
tcal@ceid:~/ $ <path_to_toolchain>/riscv64-linux-gnu-gcc -O2 -static compute_intensive.c -o cpu.riscv
tcal@ceid:~/ $ <path_to_toolchain>/riscv64-linux-gnu-gcc -O2 -static compute_intensive.c -o cpu.riscv
```

Συνιστάται η χρήση επιπέδου βελτιστοποίησης `-O2`, καθώς προσφέρει μια καλή ισορροπία μεταξύ απόδοσης και χρόνου μεταγλώττισης. Επιπλέον, η χρήση της επιλογής `-static` είναι απαραίτητη, ώστε να ενσωματωθούν όλες οι απαιτούμενες βιβλιοθήκες στο τελικό εκτελέσιμο, διασφαλίζοντας ότι μπορεί να εκτελεστεί σωστά στο περιβάλλον του gem5.

Ερώτημα 1 – Σύγκριση MinorCPU και O3CPU

Για όλα τα παρακάτω βήματα, θα εκτελέσετε τα προγράμματα και θα συγκεντρώσετε τα αποτελέσματα σε έναν ενιαίο πίνακα (όπως φαίνεται παρακάτω), χρησιμοποιώντας τα στατιστικά από το αρχείο `m5out/stats.txt` που παράγεται μετά από κάθε προσομοίωση.

		Minor CPU	O3 CPU
Κύκλοι ρολογιού	cpu.riscv		
	mem.riscv		
Εντολές	cpu.riscv		
	mem.riscv		
IPC	cpu.riscv		
	mem.riscv		
Ρυθμός αστοχίας L1I \$	cpu.riscv		
	mem.riscv		
Ρυθμός αστοχίας L1D \$	cpu.riscv		
	mem.riscv		
Ρυθμός αστοχίας L2 \$	cpu.riscv		
	mem.riscv		

Όταν εκτελείται το κάθε ένα πρόγραμμα στον προσομοιωτή, θα παρατηρήσετε ότι δημιουργείται ένας κατάλογος με όνομα «m5out» και μέσα ένα αρχείο «stats.txt». Αυτό το αρχείο περιέχει όλα τα απαραίτητα στατιστικά της εκτέλεσης του προγράμματος. Ανοίξτε το και μελετήστε την πληθώρα στατιστικών στοιχείων που παρέχονται από τον προσομοιωτή. Μπορείτε να διαπιστώσετε ότι πολλά από αυτά είστε ήδη σε θέση να τα αναγνωρίσετε.

ΥΠΟΔΕΙΞΗ

Για τη συμπλήρωση του παραπάνω πίνακα θα χρειαστεί να εντοπίσετε μετά από κάθε εκτέλεση τα εξής στατιστικά:

- board.processor.cores.core.numCycles
- simInsts
- board.processor.cores.core.ipc
- board.cache_hierarchy.l1i-cache-0.overallMissRate::total
- board.cache_hierarchy.l1d-cache-0.overallMissRate::total
- board.cache_hierarchy.l2-cache-0.overallMissRate::total

Βήμα 1

Τρέξτε το `cpu.riscv` με “--cpu minor” (διοχέτευση εντολών σε σειρά) και με “--cpu o3” (διοχέτευση εντολών εκτός σειράς) διατηρώντας σταθερές όλες τις υπόλοιπες παραμέτρους (μεγέθη κρυφών μνημών και μικροαρχιτεκτονικών δομών). Για να τρέξετε το πρόγραμμά σας στον προσομοιωτή, χρησιμοποιήστε τις παρακάτω εντολές στο root directory του gem5 για κάθε ένα από τα δύο προγράμματα που σας δίνονται:

```
tcal@ceid:~/gem5/$ ./build/RISCV/gem5.opt configs/run_riscv_se.py --cpu minor --cmd ../src_codes/cpu.riscv --options "500000"
tcal@ceid:~/gem5/$ ./build/RISCV/gem5.opt configs/run_riscv_se.py --cpu o3 --cmd ../src_codes/cpu.riscv --options "500000"
```

ΣΗΜΑΝΤΙΚΟ

Θεωρούμε ότι το configuration script βρίσκεται στον κατάλογο **configs** κάτω από το root directory του gem5, ενώ το εκτελέσιμο πρόγραμμα βρίσκεται σε έναν κατάλογο **src_codes**, ο οποίος είναι στο ίδιο επίπεδο με το root directory του gem5. Η συγκεκριμένη δομή θεωρείται δεδομένη και θα χρησιμοποιηθεί στα επόμενα βήματα της εργασίας.

Βήμα 2

Επαναλάβετε το Βήμα 1 αλλά για το εκτελέσιμο **mem.riscv**:

```
tcal@ceid:~/gem5/$ ./build/RISCV/gem5.opt configs/run_riscv_se.py --cpu minor --cmd ../src_codes/mem.riscv --options "65536 2000000"
tcal@ceid:~/gem5/$ ./build/RISCV/gem5.opt configs/run_riscv_se.py --cpu o3 --cmd ../src_codes/mem.riscv --options "65536 2000000"
```

Ερώτημα 1.1

Για ποιο από τα δύο προγράμματα η μετάβαση από MinorCPU σε O3CPU δίνει μεγαλύτερη μείωση των κύκλων εκτέλεσης;

Ερώτημα 1.2

Υπολογίστε την επιτάχυνση της O3CPU ως προς τον MinorCPU για κάθε πρόγραμμα.

Ερώτημα 1.3

Εξηγήστε γιατί το συγκεκριμένο πρόγραμμα που βρήκατε στο Ερώτημα 1.1 ωφελείται περισσότερο από την O3CPU.

Ερώτημα 1.4

Εξηγήστε γιατί το άλλο πρόγραμμα ωφελείται λιγότερο από την O3CPU.

Ερώτημα 1.5

Σε τι ποσοστό επηρεάζεται το πλήθος των εντολών που εκτελούνται σε κάθε πρόγραμμα μεταξύ των MinorCPU και O3CPU; Είναι λογική αυτή η συμπεριφορά; Αιτιολογήστε την απάντησή σας.

Ερώτημα 2 – Επίδραση της ιεραρχίας κρυφών μνημών

Σε αυτό το μέρος θα μελετήσετε πως τα ίδια προγράμματα επηρεάζονται από αλλαγή στο μέγεθος των κρυφών μνημών.

ΥΠΟΔΕΙΞΗ

Στην εντολή που χρησιμοποιείται για να τρέξετε την προσομοίωση μπορείτε να προσθέσετε τα arguments `--l1i_size L1I_SIZE --l1d_size L1D_SIZE --l2_size L2_SIZE`, όπου `L1I_SIZE`, `L1D_SIZE`, `L2_SIZE` τα μεγέθη των κρυφών μνημών πρώτου επιπέδου κρυφής μνήμης εντολών, πρώτου επιπέδου κρυφής μνήμης δεδομένων και δευτέρου επιπέδου, αντίστοιχα, που θέλετε κάθε φορά να έχει η διαμόρφωση της προσομοίωσης.

Για όλα τα παρακάτω βήματα, θα εκτελέσετε τα προγράμματα και θα συγκεντρώσετε τα αποτελέσματα σε έναν ενιαίο πίνακα (όπως φαίνεται παρακάτω), χρησιμοποιώντας τα στατιστικά από το αρχείο `m5out/stats.txt` που παράγεται μετά από κάθε προσομοίωση.

		Minor CPU				O3 CPU			
		A	B	C	D	A	B	C	D
Κύκλοι ρολογιού	cpu.riscv								
	mem.riscv								
Εντολές	cpu.riscv								
	mem.riscv								
IPC	cpu.riscv								
	mem.riscv								
Ρυθμός αστοχίας L1I \$	cpu.riscv								
	mem.riscv								
Ρυθμός αστοχίας L1D \$	cpu.riscv								
	mem.riscv								
Ρυθμός αστοχίας L2 \$	cpu.riscv								
	mem.riscv								

Βήμα 1

Για το `cpu.riscv`, τρέξτε ξανά σε MinorCPU και O3CPU, αλλά τώρα με τέσσερις διαφορετικές διαμορφώσεις/μεγέθη των κρυφών μνημών (caches).

1. Διαμόρφωση Cache A: L1I = 16KiB, L1D = 16KiB, L2 = 256KiB
2. Διαμόρφωση Cache B: L1I = 16KiB, L1D = 16KiB, L2 = 64KiB
3. Διαμόρφωση Cache C: L1I = 8KiB, L1D = 8KiB, L2 = 1MiB
4. Διαμόρφωση Cache D: L1I = 32KiB, L1D = 32KiB, L2 = 1MiB

Βήμα 2

Για το `mem.riscv` επαναλάβετε τις ίδιες εκτελέσεις με τις ίδιες δύο διαμορφώσεις κρυφών μνημών.

Ερώτημα 2.1

Ποιο πρόγραμμα επηρεάζεται περισσότερο από τις αλλαγές στα μεγέθη των κρυφών μνημών;

Ερώτημα 2.2

Εξηγήστε γιατί το άλλο πρόγραμμα επηρεάζεται λιγότερο.

Ερώτημα 2.3

Για κάθε πρόγραμμα να συγκρίνετε τις αλλαγές στους κύκλους ρολογιού με τις αλλαγές στους ρυθμούς αστοχίας των κρυφών μνημών. Συμφωνούν μεταξύ τους τα αποτελέσματα; Εξηγήστε σύντομα.

Ερώτημα 3 – Επίδραση του ROB στον O3CPU

Σε αυτό το μέρος θα αλλάξετε μόνο τον αριθμό entries του ROB στην O3CPU. Θα χρησιμοποιήσετε μόνο O3CPU (αφού άλλωστε δεν υπάρχει η έννοια του reorder buffer σε διοχετεύσεις εντολών με εκτέλεση σε σειρά – in-order) και την default οργάνωση των κρυφών μνημών (χωρίς τις αλλαγές που κάναμε στο Ερώτημα 2).

ΥΠΟΔΕΙΞΗ

Στην εντολή που χρησιμοποιείται για να τρέξετε την προσομοίωση μπορείτε να προσθέσετε το argument `--rob_entries ROB_ENTRIES`, όπου ROB_ENTRIES η τιμή των entries που θέλετε κάθε φορά να έχει η διαμόρφωση της προσομοίωσης.

Για όλα τα παρακάτω βήματα, θα εκτελέσετε τα προγράμματα και θα συγκεντρώσετε τα αποτελέσματα σε έναν ενιαίο πίνακα (όπως φαίνεται παρακάτω), χρησιμοποιώντας τα στατιστικά από το αρχείο `m5out/stats.txt` που παράγεται μετά από κάθε προσομοίωση.

		O3 CPU			
		Default	ROB A	ROB B	ROB C
Κύκλοι ρολογιού	cpu.riscv				
	mem.riscv				
Εντολές	cpu.riscv				
	mem.riscv				
IPC	cpu.riscv				
	mem.riscv				
«Χαμένοι» κύκλοι (idle cycles)	cpu.riscv				
	mem.riscv				
Full ROB Events	cpu.riscv				
	mem.riscv				
Full Register Events	cpu.riscv				
	mem.riscv				

ΥΠΟΔΕΙΞΗ

Τα τρία νέα στατιστικά μπορείτε να τα βρείτε με τα εξής ονόματα μέσα στο αρχείο m5out/stats.txt:

- board.processor.cores.core.idleCycles
- board.processor.cores.core.rename.ROBFullEvents
- board.processor.cores.core.rename.fullRegistersEvents

Βήμα 1

Τρέξτε το `cpu.riscv` με O3CPU και με:

1. Default = 192 entries
2. ROB A = 32 entries
3. ROB B = 48 entries
4. ROB C = 96 entries

Κρατήστε όλες τις άλλες παραμέτρους σταθερές.

Βήμα 2

Τρέξτε το `mem.riscv` με O3CPU και με:

1. Default = 192 entries
2. ROB A = 32 entries
3. ROB B = 48 entries
4. ROB C = 96 entries

Κρατήστε όλες τις άλλες παραμέτρους σταθερές.

Ερώτημα 3.1

Εξηγήστε τι υποδηλώνουν τα νέα στατιστικά που προσθέσαμε σε αυτό το ερώτημα και γιατί είναι σημαντική η ερμηνεία τους.

Ερώτημα 3.2

Για ποιο πρόγραμμα η αύξηση του μεγέθους του ROB οδηγεί στη μεγαλύτερη μείωση των κύκλων;

Ερώτημα 3.3

Γιατί η αύξηση του μεγέθους του ROB βοηθάει περισσότερο σε αυτό το πρόγραμμα; Είναι λογική αυτή η συμπεριφορά; Αιτιολογήστε την απάντησή σας.

Ερώτημα 4 – Μελέτη φυσικών ακέραιων καταχωρητών

Σε αυτό το μέρος θα αλλάξετε μόνο τον αριθμό των φυσικών ακέραιων καταχωρητών του O3CPU. Θα χρησιμοποιήσετε ξανά μόνο O3CPU και την default οργάνωση των κρυφών μνημών και μεγέθους ROB (χωρίς τις αλλαγές που κάναμε στο Ερώτημα 2 και το Ερώτημα 3).

ΥΠΟΔΕΙΞΗ

Στην εντολή που χρησιμοποιείται για να τρέξετε την προσομοίωση μπορείτε να προσθέσετε το argument `--num_phys_int_regs NUM_PHYS_INT_REGS`, όπου NUM_PHYS_INT_REGS το πλήθος των φυσικών καταχωρητών που θέλετε κάθε φορά να έχει η διαμόρφωση της προσομοίωσης.

Για όλα τα παρακάτω βήματα, θα εκτελέσετε τα προγράμματα και θα συγκεντρώσετε τα αποτελέσματα σε έναν ενιαίο πίνακα (όπως φαίνεται παρακάτω), χρησιμοποιώντας τα στατιστικά από το αρχείο `m5out/stats.txt` που παράγεται μετά από κάθε προσομοίωση.

		O3 CPU			
		Default	PRF A	PRF B	PRF C
Κύκλοι ρολογιού	cpu.riscv				
	mem.riscv				
Εντολές	cpu.riscv				
	mem.riscv				
IPC	cpu.riscv				
	mem.riscv				
«Χαμένοι» κύκλοι (idle cycles)	cpu.riscv				
	mem.riscv				
Full ROB Events	cpu.riscv				
	mem.riscv				
Full Register Events	cpu.riscv				
	mem.riscv				

Βήμα 1

Τρέξτε το `cpu.riscv` με O3CPU και με:

1. Default = 256 entries
2. PRF (Physical Register File) A = 64 καταχωρητές
3. PRF (Physical Register File) B = 128 καταχωρητές
4. PRF (Physical Register File) C = 192 καταχωρητές

Κρατήστε όλες τις άλλες παραμέτρους σταθερές.

Βήμα 2

Τρέξτε το `mem.riscv` με O3CPU και με:

1. Default = 256 entries
2. PRF (Physical Register File) A = 64 καταχωρητές
3. PRF (Physical Register File) B = 128 καταχωρητές
4. PRF (Physical Register File) C = 192 καταχωρητές

Κρατήστε όλες τις άλλες παραμέτρους σταθερές.

Ερώτημα 4.1

Ποιο πρόγραμμα επηρεάζεται περισσότερο από την αύξηση των φυσικών καταχωρητών; Δώστε μια σύντομη εξήγηση για το γιατί.

Ερώτημα 5 – Τελικό συμπέρασμα

Ποιο είναι το βασικό συμπέρασμα της εργασίας για τη σχέση ανάμεσα στο πρόγραμμα και στη μικροαρχιτεκτονική;

Ερώτημα 6 (εκτός ύλης 😊)

Στο πρόγραμμα `memory_intensive.c` που σας δίνεται, ο χώρος μνήμης που δεσμεύεται με `malloc()` στην αρχή δεν απελευθερώνεται στο τέλος της εκτέλεσης. Θεωρείτε ότι αυτό είναι σωστό ή λάθος; Αιτιολογήστε την απάντησή σας.

Πληροφορίες και οδηγίες υποβολής της εργασίας

1. Η εργασία αποτελεί το 15% του βαθμού του μαθήματος.
2. Κάθε ομάδα μπορεί να αποτελείται από 1 ή 2 άτομα και **πρέπει να έχετε δηλώσει το όνομά σας στην αντίστοιχη ομάδα στο eClass.**
3. Στο eClass παραδίδετε ένα μοναδικό αρχείο σε μορφή PDF (αποκλειστικά) που θα περιέχει τις απαντήσεις όλων των ερωτημάτων και τεκμηρίωση των απαντήσεων της άσκησης. Σε αυτό το αρχείο θα πρέπει επίσης να έχετε συμπληρώσει τα στοιχεία όλων των μελών της ομάδας (Ονοματεπώνυμο ΚΑΙ Αριθμό Μητρώου). Στο eClass αρκεί να ανεβάσει τις απαντήσεις μόνο ένα μέλος της ομάδας – δε χρειάζεται να ανεβάσετε πολλές φορές το ίδιο PDF.
4. Όλα τα μέλη της ομάδας πρέπει να έχουν ισότιμη συμμετοχή και να γνωρίζουν τις λεπτομέρειες της υλοποίησης της εργασίας.
5. Η εργασία θα παρουσιαστεί στο εργαστήριο την Τετάρτη 6/5/2026.
6. Για την εξεταστική Σεπτεμβρίου ΔΕΝ θα δοθούν άλλες εργασίες. Τον Σεπτέμβριο εξετάζεται μόνο το γραπτό αλλά ο βαθμός της εργασίας διατηρείται για την επαναληπτική εξέταση του Σεπτεμβρίου του ίδιου Ακαδημαϊκού Έτους. **Βαθμοί εργασιών ΔΕΝ διατηρούνται για επόμενα Ακαδημαϊκά Έτη.**